



University of Science and Technology, Chittagong (USTC)
Department of Computer Science and Engineering

LAB REPORT-02

Course Title: Computer Graphics and Image Processing Lab

Lab Course Code: CSE 418

Submitted to:

Umme Aiman Jannat
Lecturer,
CSE, FSET, USTC

Submitted by:

Name: Upama Bose
Id: 0022320005101078
Reg. No: 1084
Semester: 7th

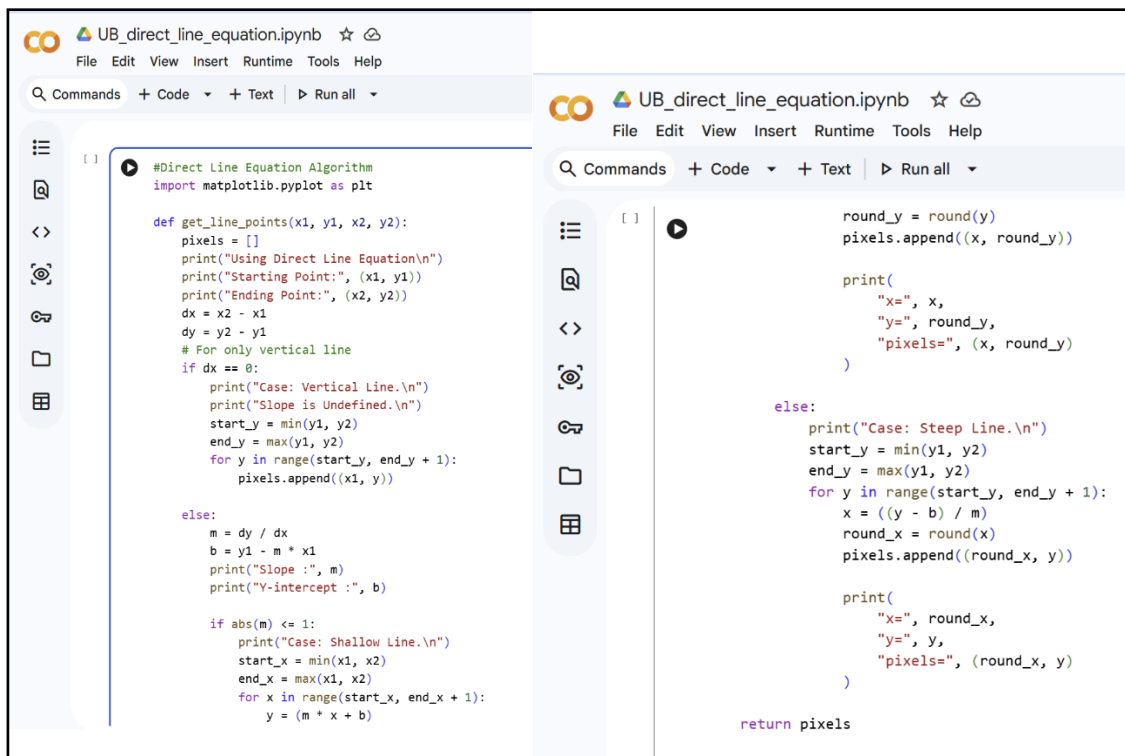
Experiment No: 01

Experiment Name: Implementation of Direct Line Equation Algorithm for Line Drawing using Python.

Objective

To understand basic concept of line drawing in computer graphics and implement the Direct Line Equation Algorithm.

Implementation



```
#Direct Line Equation Algorithm
import matplotlib.pyplot as plt

def get_line_points(x1, y1, x2, y2):
    pixels = []
    print("Using Direct Line Equation\n")
    print("Starting Point:", (x1, y1))
    print("Ending Point:", (x2, y2))
    dx = x2 - x1
    dy = y2 - y1
    # For only vertical line
    if dx == 0:
        print("Case: Vertical Line.\n")
        print("Slope is Undefined.\n")
        start_y = min(y1, y2)
        end_y = max(y1, y2)
        for y in range(start_y, end_y + 1):
            pixels.append((x1, y))
    else:
        m = dy / dx
        b = y1 - m * x1
        print("Slope :", m)
        print("V-intercept :", b)

        if abs(m) <= 1:
            print("Case: Shallow Line.\n")
            start_x = min(x1, x2)
            end_x = max(x1, x2)
            for x in range(start_x, end_x + 1):
                y = (m * x + b)

        else:
            print("Case: Steep Line.\n")
            start_y = min(y1, y2)
            end_y = max(y1, y2)
            for y in range(start_y, end_y + 1):
                x = ((y - b) / m)
                round_x = round(x)
                pixels.append((round_x, y))

        print(
            "x=", round_x,
            "y=", y,
            "pixels=", (round_x, y)
        )

    return pixels
```

```
def plot_line(pixels, x1, y1, x2, y2):
    xs = [p[0] for p in pixels]
    ys = [p[1] for p in pixels]

    plt.figure(figsize=(7, 6))
    plt.plot(xs, ys, color="blue", linewidth=1, label="Line")
    plt.scatter(xs, ys, color="red", s=25, zorder=5, label="Computed points")
    plt.scatter([x1, x2], [y1, y2], color="green", s=80,
                marker="o", zorder=6, label="Start / End")

    plt.title("Line Plot using Direct Line Equation (y = mx + b)")
    plt.xlabel("X-axis")
    plt.ylabel("Y-axis")
    plt.axhline(0, color="gray", linewidth=0.5)
    plt.axvline(0, color="gray", linewidth=0.5)
    plt.grid(True, linestyle="--", alpha=0.6)
    plt.legend()
    plt.axis("equal")
    plt.savefig("line_plot.png", dpi=150)
    print("\nPlot saved as 'line_plot.png'")
    plt.show()

def main():
    print("Enter the starting point:")
    x1 = int(input(" x1 = "))
    y1 = int(input(" y1 = "))

    print("Enter the ending point:")
    x2 = int(input(" x2 = "))
    y2 = int(input(" y2 = "))

    pixels = get_line_points(x1, y1, x2, y2)
    plot_line(pixels, x1, y1, x2, y2)

if __name__ == "__main__":
    main()
```

Output

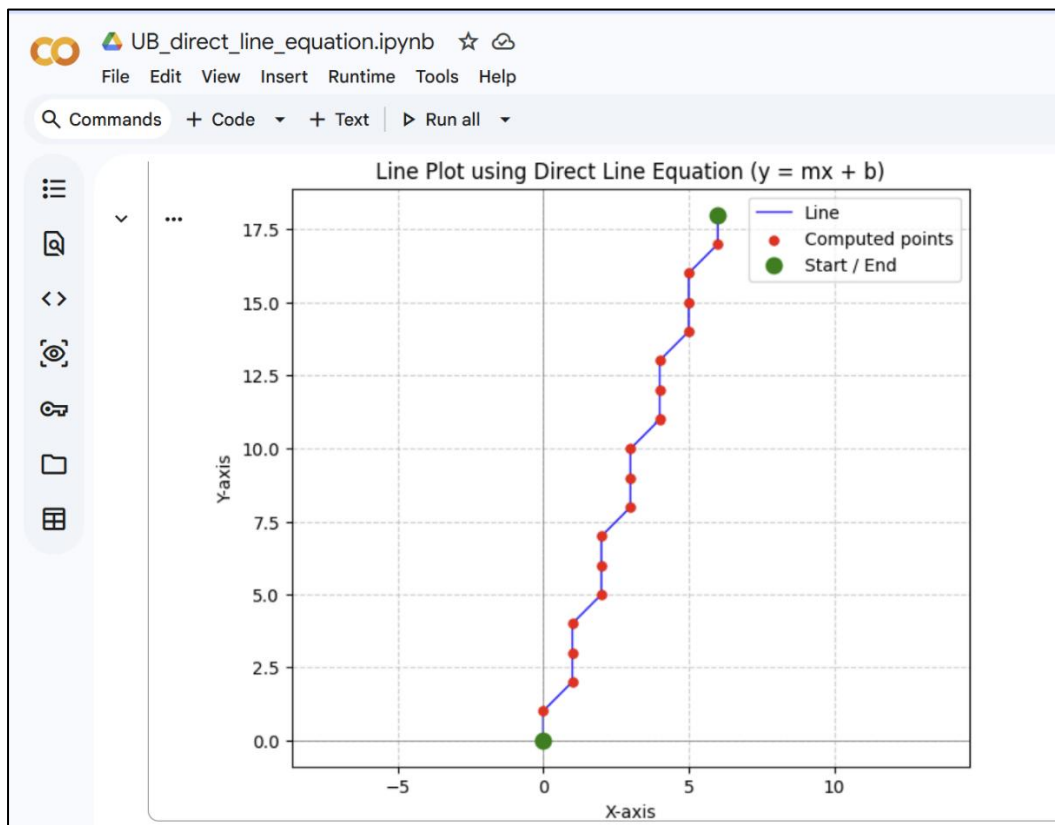
```
UB_direct_line_equation.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

Enter the starting point:
x1 = 0
y1 = 0
Enter the ending point:
x2 = 6
y2 = 18
Using Direct Line Equation

Starting Point: (0, 0)
Ending Point: (6, 18)
Slope : 3.0
Y-intercept : 0.0
Case: Steep Line.

x= 0 y= 0 pixels= (0, 0)
x= 0 y= 1 pixels= (0, 1)
x= 1 y= 2 pixels= (1, 2)
x= 1 y= 3 pixels= (1, 3)
x= 1 y= 4 pixels= (1, 4)
x= 2 y= 5 pixels= (2, 5)
x= 2 y= 6 pixels= (2, 6)
x= 2 y= 7 pixels= (2, 7)
x= 3 y= 8 pixels= (3, 8)
x= 3 y= 9 pixels= (3, 9)
x= 3 y= 10 pixels= (3, 10)
x= 4 y= 11 pixels= (4, 11)
x= 4 y= 12 pixels= (4, 12)
x= 4 y= 13 pixels= (4, 13)
x= 5 y= 14 pixels= (5, 14)
x= 5 y= 15 pixels= (5, 15)
x= 5 y= 16 pixels= (5, 16)
x= 6 y= 17 pixels= (6, 17)
x= 6 y= 18 pixels= (6, 18)
```



Experiment No: 02

Experiment Name: Implementation of Digital Differential Analyzer (DDA) Line Drawing Algorithm using Python.

Objective

To understand the DDA line drawing algorithm and implement DDA using Python.

Implementation

```
UB_DDA_lineDrawingAlgorithm.py
File Edit View Insert Runtime Tools Help

[1] 0s
# DDA Line Drawing Algorithm
import matplotlib.pyplot as plt

def get_line_points(x1, y1, x2, y2):
    pixels = []

    print("Using DDA Line Drawing Algorithm\n")
    print("Starting Point:", (x1, y1))
    print("Ending Point:", (x2, y2))

    dx = x2 - x1
    dy = y2 - y1

    print("dx =", dx)
    print("dy =", dy)

    # Calculate number of steps
    steps = max(abs(dx), abs(dy))

    print("Number of Steps:", steps)

    # If both points are the same
    if steps == 0:
        print("\nCase: Single Point.")
        pixels.append((x1, y1))
        print("pixels =", (x1, y1))
        return pixels

    # Calculate increments
    x_increment = dx / steps
    y_increment = dy / steps

    print("X Increment:", x_increment)
    print("Y Increment:", y_increment)

    # Starting point
    x = x1
    y = y1

    print("\nGenerated Pixels:\n")

    for i in range(steps + 1):
        round_x = round(x)
        round_y = round(y)

        pixels.append((round_x, round_y))

        print(
            "Step =", i,
            "x =", round_x,
            "y =", round_y,
            "pixels =", (round_x, round_y)
        )

        # Update x and y
        x = x + x_increment
        y = y + y_increment

    return pixels
```

```
UB_DDA_lineDrawingAlgorithm.py
File Edit View Insert Runtime Tools Help

[2] 0s
def plot_line(pixels, x1, y1, x2, y2):
    xs = [p[0] for p in pixels]
    ys = [p[1] for p in pixels]

    plt.figure(figsize=(7, 6))
    # Draw line connecting the pixels
    plt.plot(
        xs,
        ys,
        color="blue",
        linewidth=1,
        label="DDA Line"
    )

    # Show calculated pixels
    plt.scatter(
        xs,
        ys,
        color="red",
        s=35,
        zorder=5,
        label="Computed Pixels"
    )

    # Show starting and ending points
    plt.scatter(
        [x1, x2],
        [y1, y2],
        color="green",
        s=100,
        marker="o",
        zorder=6,
        label="Start / End"
    )

    plt.title("Line Drawing using DDA Algorithm")
    plt.xlabel("X-axis")
    plt.ylabel("Y-axis")

    plt.axhline(
        0,
        color="gray",
        linewidth=0.5
    )

    plt.axvline(
        0,
        color="gray",
        linewidth=0.5
    )

    plt.grid(
        True,
        linestyle="--",
        alpha=0.6
    )

    plt.legend()
    plt.axis("equal")

    plt.savefig(
        "dda_line_plot.png",
        dpi=150
    )

    print("\nPlot saved as 'dda_line_plot.png'")
    plt.show()
```

```
UB_DDA_lineDrawingAlgorithm.ipynb
File Edit View Insert Runtime Tools Help

[3] ✓ 45s def main():

    print("Enter the starting point:")
    x1 = int(input(" x1 = "))
    y1 = int(input(" y1 = "))

    print("\nEnter the ending point:")
    x2 = int(input(" x2 = "))
    y2 = int(input(" y2 = "))

    pixels = get_line_points(
        x1, y1, x2, y2
    )

    plot_line(
        pixels,
        x1, y1,
        x2, y2
    )

if __name__ == "__main__":
    main()
```

Output

```
UB_DDA_lineDrawingAlgorithm.ipynb
File Edit View Insert Runtime Tools Help

Enter the starting point:
x1 = 0
y1 = 0

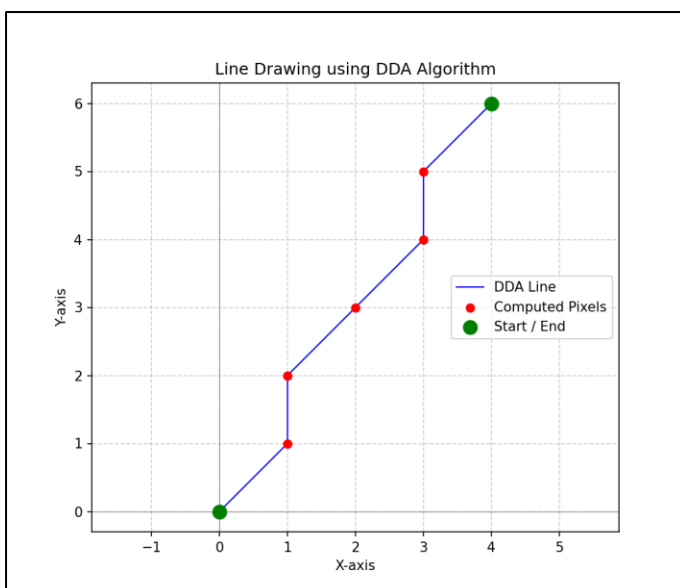
Enter the ending point:
x2 = 4
y2 = 6

Using DDA Line Drawing Algorithm

Starting Point: (0, 0)
Ending Point: (4, 6)
dx = 4
dy = 6
Number of Steps: 6
X Increment: 0.6666666666666666
Y Increment: 1.0

Generated Pixels:

Step = 0 x = 0 y = 0 pixels = (0, 0)
Step = 1 x = 1 y = 1 pixels = (1, 1)
Step = 2 x = 1 y = 2 pixels = (1, 2)
Step = 3 x = 2 y = 3 pixels = (2, 3)
Step = 4 x = 3 y = 4 pixels = (3, 4)
Step = 5 x = 3 y = 5 pixels = (3, 5)
Step = 6 x = 4 y = 6 pixels = (4, 6)
```



Experiment No: 03

Experiment Name: Implementation of Bresenham's Line Drawing Algorithm using Python.

Objective

To understand and implement Bresenham's algorithm using Python.

Implementation

```
UB_Bresenham's_algorithm.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all

# Bresenham's Line Drawing Algorithm
import matplotlib.pyplot as plt
def get_line_points(x1, y1, x2, y2):
    pixels = []

    print("Using Bresenham's Line Drawing Algorithm\n")
    print("Starting Point:", (x1, y1))
    print("Ending Point:", (x2, y2))
    dx = x2 - x1
    dy = y2 - y1

    print("dx =", dx)
    print("dy =", dy)

    # horizontal line
    if dy == 0:
        print("\nCase: Horizontal Line.")

        start_x = min(x1, x2)
        end_x = max(x1, x2)

        for x in range(start_x, end_x + 1):
            pixels.append((x, y1))

        print(
            "x =", x,
            "y =", y1,
            "pixels =", (x, y1)
        )

        return pixels

    # vertical line
    if dx == 0:
        print("\nCase: Vertical Line.")

        start_y = min(y1, y2)
        end_y = max(y1, y2)

        for y in range(start_y, end_y + 1):
            pixels.append((x1, y))

        print(
            "x =", x1,
            "y =", y,
            "pixels =", (x1, y)
        )

        return pixels

    # Bresenham's for positive slope
    if dy > 0:
        # Shallow line: 0 < slope < 1
        if abs(dy) <= abs(dx):
            print("\nCase: Positive Shallow Line.")
            print("Condition: |dy| <= |dx|")

            #x increase from left to right
            if x1 > x2:
                x1, x2 = x2, x1
                y1, y2 = y2, y1

            dx = x2 - x1
            dy = y2 - y1

            p = 2 * dy - dx

            print("Initial Decision Parameter:", p)
            print()

            x = x1
            y = y1

            for i in range(dx + 1):
                pixels.append((x, y))

                print(
                    "Step =", i,
                    "x =", x,
                    "y =", y,
                    "p =", p,
                    "pixels =", (x, y)
                )

                if p < 0:
                    #to choose East pixel
                    p = p + 2 * dy

                    print(" Decision: Choose E")
                    print(" New p =", p)

                else:
                    #to choose North-East pixel
                    y = y + 1
                    p = p + 2 * dy - 2 * dx

                    print(" Decision: Choose NE")
                    print(" New p =", p)

                x = x + 1

            # Steep line: slope > 1
            else:
                print("\nCase: Positive Steep Line.")
                print("Condition: |dy| > |dx|")

                #y increase from bottom to top
                if y1 > y2:
                    x1, x2 = x2, x1
                    y1, y2 = y2, y1

                dx = x2 - x1
                dy = y2 - y1

                p = 2 * dx - dy

                print("Initial Decision Parameter:", p)
                print()

                x = x1
                y = y1

                for i in range(dy + 1):
                    pixels.append((x, y))

                    print(
                        "Step =", i,
                        "x =", x,
                        "y =", y,
                        "p =", p,
                        "pixels =", (x, y)
                    )

                    if p < 0:
                        p = p + 2 * dx

                        print(" Decision: Choose N")
                        print(" New p =", p)

                    else:
                        # Choose North-East pixel
                        x = x + 1
                        p = p + 2 * dx - 2 * dy

                        print(" Decision: Choose NE")
                        print(" New p =", p)

                    y = y + 1

                return pixels

            # Bresenham for negative slope
            else:
                # Shallow negative line
                if abs(dy) <= abs(dx):
                    print("\nCase: Negative Shallow Line.")
                    print("Condition: |dy| <= |dx|")

                    #x increase from left to right
                    if x1 > x2:
                        x1, x2 = x2, x1
                        y1, y2 = y2, y1

                    dx = x2 - x1
                    dy = abs(y2 - y1)

                    p = 2 * dy - dx

                    print("Initial Decision Parameter:", p)
                    print()

                    x = x1
                    y = y1

                    #to determine direction of y
                    y_step = 1 if y2 > y1 else -1

                    for i in range(dx + 1):
                        pixels.append((x, y))

                        print(
                            "Step =", i,
                            "x =", x,
                            "y =", y,
                            "p =", p,
                            "pixels =", (x, y)
                        )

                        if p < 0:
                            p = p + 2 * dx

                            print(" Decision: Choose S")
                            print(" New p =", p)

                        else:
                            x = x + x_step
                            p = p + 2 * dx - 2 * dy

                            print(" Decision: Choose SW")
                            print(" New p =", p)

                            y = y + y_step

                    return pixels

            #to determine direction of x
            x_step = 1 if x2 > x1 else -1

            for i in range(dy + 1):
                pixels.append((x, y))

                print(
                    "Step =", i,
                    "x =", x,
                    "y =", y,
                    "p =", p,
                    "pixels =", (x, y)
                )

                if p < 0:
                    p = p + 2 * dy

                    print(" Decision: Choose SE")
                    print(" New p =", p)

                else:
                    x = x + x_step
                    p = p + 2 * dy - 2 * dx

                    print(" Decision: Choose SW")
                    print(" New p =", p)

                    y = y + y_step

            return pixels
```

```
UB_Bresenham's_algorithm.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all

[1] def plot_line(pixels, x1, y1, x2, y2):
    xs = [p[0] for p in pixels]
    ys = [p[1] for p in pixels]
    plt.figure(figsize=(7, 6))
    # Draw line
    plt.plot(
        xs,
        ys,
        color="blue",
        linewidth=1,
        label="Bresenham Line"
    )
    # Show calculated pixels
    plt.scatter(
        xs,
        ys,
        color="red",
        s=35,
        zorder=5,
        label="Computed Pixels"
    )
    # Show start and end points
    plt.scatter(
        [x1, x2],
        [y1, y2],
        color="green",
        s=100,
        marker="o",
        zorder=6,
        label="Start / End"
    )
    plt.title("Line Drawing using Bresenham's Algorithm")

plt.xlabel("X-axis")
plt.ylabel("Y-axis")

plt.axhline(
    0,
    color="gray",
    linewidth=0.5
)
plt.axvline(
    0,
    color="gray",
    linewidth=0.5
)
plt.grid(
    True,
    linestyle="--",
    alpha=0.6
)
plt.legend()
plt.axis("equal")
plt.savefig(
    "bresenham_line_plot.png",
    dpi=150
)
print("\nPlot saved as 'bresenham_line_plot.png'")
plt.show()

UB_Bresenham's_algorithm.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all

[4] ✓ 21s def main():
    print("Enter the starting point:")
    x1 = int(input(" x1 = "))
    y1 = int(input(" y1 = "))

    print("\nEnter the ending point:")
    x2 = int(input(" x2 = "))
    y2 = int(input(" y2 = "))

    pixels = get_line_points(
        x1, y1, x2, y2
    )

    plot_line(
        pixels,
        x1, y1,
        x2, y2
    )

if __name__ == "__main__":
    main()
```

Output

```
UB_Bresenham's_algorithm.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all

Enter the starting point:
x1 = 9
y1 = 18

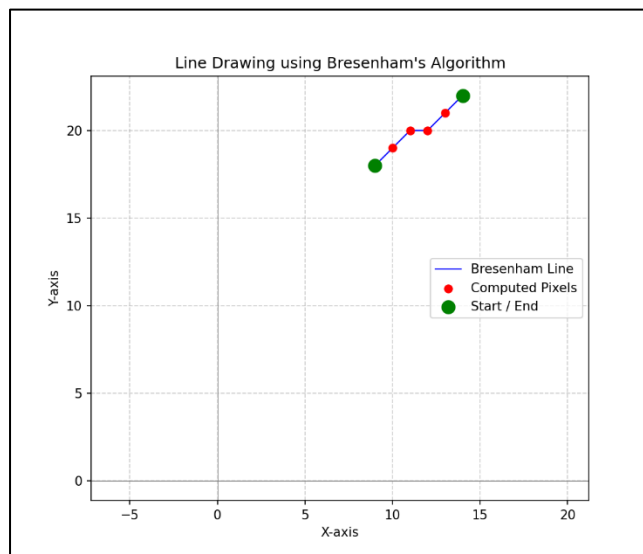
Enter the ending point:
x2 = 14
y2 = 22

Using Bresenham's Line Drawing Algorithm

Starting Point: (9, 18)
Ending Point: (14, 22)
dx = 5
dy = 4

Case: Positive Shallow Line.
Condition: |dy| <= |dx|
Initial Decision Parameter: 3

Step = 0 x = 9 y = 18 p = 3 pixels = (9, 18)
Decision: Choose NE
New p = 1
Step = 1 x = 10 y = 19 p = 1 pixels = (10, 19)
Decision: Choose NE
New p = -1
Step = 2 x = 11 y = 20 p = -1 pixels = (11, 20)
Decision: Choose E
New p = 7
Step = 3 x = 12 y = 20 p = 7 pixels = (12, 20)
Decision: Choose NE
New p = 5
Step = 4 x = 13 y = 21 p = 5 pixels = (13, 21)
Decision: Choose NE
New p = 3
Step = 5 x = 14 y = 22 p = 3 pixels = (14, 22)
Decision: Choose NE
New p = 1
```



Experiment No: 04

Experiment Name: Implementation of Bresenham's Circle Drawing Algorithm using Python.

Objective

To understand and implement Bresenham's circle drawing algorithm using Python.

Implementation

```
UB_Bresenham's Circle Drawing Algorithm.ipynb
File Edit View Insert Runtime Tools Help

[10] ✓ Gs
# Bresenham's Circle Drawing Algorithm
import matplotlib.pyplot as plt
def get_circle_points(xc, yc, r):
    pixels = []
    print("Using Bresenham's Circle Drawing Algorithm\n")
    print("Center Point:", (xc, yc))
    print("Radius:", r)
    # Initial values
    xi = 0
    yi = r
    # Initial decision parameter
    p = 3 - 2 * r
    print("Initial xi =", xi)
    print("Initial yi =", yi)
    print("Initial Decision Parameter:", p)
    print()
    step = 0
    while xi <= yi:
        #to store current values
        current_x = xi
        current_y = yi
        current_p = p
        print(
            "Step =", step,
            "xi =", current_x,
            "yi =", current_y,
            "p =", current_p
        )
        # Case 1: p < 0
        if p < 0:
            xi_next = xi + 1
            yi_next = yi
            p_next = p + 4 * xi + 6
            print(" Decision: Choose E")
            print(
                " xi+1 =", xi_next,
                "yi+1 =", yi_next
            )
            print(
                " Formula: p(i+1) = p(i) + 4xi + 6"
            )
            # Case 2: p >= 0
            else:
                xi_next = xi + 1
                yi_next = yi - 1
                p_next = p + 4 * (xi - yi) + 10
                print(" Decision: Choose SE")
                print(
                    " xi+1 =", xi_next,
                    "yi+1 =", yi_next
                )
                print(
                    " Formula: p(i+1) = p(i) + 4(xi - yi) + 10"
                )
                #to generate 8 symmetric pixels
                symmetric_points = [
                    (xc + current_x, yc + current_y),
                    (xc - current_x, yc + current_y),
                    (xc + current_x, yc - current_y),
                    (xc - current_x, yc - current_y),
                    (xc + current_y, yc + current_x),
                    (xc - current_y, yc + current_x),
                    (xc + current_y, yc - current_x),
                    (xc - current_y, yc - current_x)
                ]
                print(" 8 Symmetric Pixels:")
                for point in symmetric_points:
                    if point not in pixels:
                        pixels.append(point)
                print("    ", point)
            print()
        # Update values
        xi = xi_next
        yi = yi_next
        p = p_next
        step = step + 1
    return pixels

plt.gca().add_patch(circle)
#to draw radius
plt.plot(
    [xc, xc + r],
    [yc, yc],
    color="black",
    linestyle="--",
    linewidth=1,
    label="Radius"
)
plt.title(
    "Circle Drawing using Bresenham's Algorithm"
)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.axhline(
    0,
    color="gray",
    linewidth=0.5
)
plt.axvline(
    0,
    color="gray",
    linewidth=0.5
)
plt.grid(
    True,
    linestyle="--",
    alpha=0.6
)
plt.legend()
plt.axis("equal")

plt.savefig(
    "bresenham_circle_plot.png",
    dpi=150
)
print(
    "\nPlot saved as 'bresenham_circle_plot.png'"
)
plt.show()

def main():
    print("Enter the center of the circle:")
    xc = int(input(" xc = "))
    yc = int(input(" yc = "))
    print("\nEnter the radius:")
    r = int(input(" r = "))
    # Check radius
    if r < 0:
        print("\nRadius cannot be negative.")
        return
    pixels = get_circle_points(
        xc,
        yc,
        r
    )
    plot_circle(
        pixels,
        xc,
        yc,
        r
    )
if __name__ == "__main__":
    main()
```

```
def plot_circle(pixels, xc, yc, r):
    #get x and y coordinates
    xs = [p[0] for p in pixels]
    ys = [p[1] for p in pixels]
    plt.figure(figsize=(7, 7))
    #to draw calculated pixels
    plt.scatter(
        xs,
        ys,
        color="red",
        s=35,
        zorder=5,
        label="Computed Pixels"
    )
    #to draw center point
    plt.scatter(
        [xc],
        [yc],
        color="green",
        s=100,
        marker="o",
        zorder=6,
        label="Center"
    )
    #to draw ideal circle
    circle = plt.Circle(
        (xc, yc),
        r,
        fill=False,
        color="blue",
        linewidth=1.5,
        label="Bresenham Circle"
    )
    plt.gca().add_patch(circle)
    #to draw radius
    plt.plot(
        [xc, xc + r],
        [yc, yc],
        color="black",
        linestyle="--",
        linewidth=1,
        label="Radius"
    )
    plt.title(
        "Circle Drawing using Bresenham's Algorithm"
    )
    plt.xlabel("X-axis")
    plt.ylabel("Y-axis")
    plt.axhline(
        0,
        color="gray",
        linewidth=0.5
    )
    plt.axvline(
        0,
        color="gray",
        linewidth=0.5
    )
    plt.grid(
        True,
        linestyle="--",
        alpha=0.6
    )
    plt.legend()
    plt.axis("equal")

plt.savefig(
    "bresenham_circle_plot.png",
    dpi=150
)
print(
    "\nPlot saved as 'bresenham_circle_plot.png'"
)
plt.show()

def main():
    print("Enter the center of the circle:")
    xc = int(input(" xc = "))
    yc = int(input(" yc = "))
    print("\nEnter the radius:")
    r = int(input(" r = "))
    # Check radius
    if r < 0:
        print("\nRadius cannot be negative.")
        return
    pixels = get_circle_points(
        xc,
        yc,
        r
    )
    plot_circle(
        pixels,
        xc,
        yc,
        r
    )
if __name__ == "__main__":
    main()
```

Output

```

Enter the center of the circle:
... xc = 0
    yc = 0

Enter the radius:
r = 6
Using Bresenham's Circle Drawing Algorithm ...

Center Point: (0, 0)
Radius: 6
Initial xi = 0
Initial yi = 6
Initial Decision Parameter: -9

Step = 0 xi = 0 yi = 6 p = -9
Decision: Choose E
xi+1 = 1 yi+1 = 6
Formula: p(i+1) = p(i) + 4xi + 6
New p = -3
8 Symmetric Pixels:
(0, 6)
(0, 6)
(0, -6)
(0, -6)
(6, 0)
(-6, 0)
(6, 0)
(-6, 0)

Step = 1 xi = 1 yi = 6 p = -3
Decision: Choose E
xi+1 = 2 yi+1 = 6
Formula: p(i+1) = p(i) + 4xi + 6
New p = 7
8 Symmetric Pixels:
(1, 6)
(-1, 6)
(1, -6)
(-1, -6)
(6, 1)
(-6, 1)
(6, -1)
(-6, -1)

Step = 2 xi = 2 yi = 6 p = 7
Decision: Choose SE
xi+1 = 3 yi+1 = 5
Formula: p(i+1) = p(i) + 4(xi - yi) + 10
New p = 1
8 Symmetric Pixels:
(2, 6)
(-2, 6)
(2, -6)
(-2, -6)
(6, 2)
(-6, 2)
(6, -2)
(-6, -2)

Step = 3 xi = 3 yi = 5 p = 1
Decision: Choose SE
xi+1 = 4 yi+1 = 4
Formula: p(i+1) = p(i) + 4(xi - yi) + 10
New p = 13
8 Symmetric Pixels:
(3, 5)
(-3, 5)
(3, -5)
(-3, -5)
(5, 3)
(-5, 3)
(5, -3)
(-5, -3)

Step = 4 xi = 4 yi = 4 p = 3
Decision: Choose SE
xi+1 = 5 yi+1 = 3
Formula: p(i+1) = p(i) + 4(xi - yi) + 10
New p = 13
8 Symmetric Pixels:
(4, 4)
(-4, 4)
(4, -4)
(-4, -4)
(4, 4)
(-4, 4)
(4, -4)
(-4, -4)

```

